

任务一前置说明

1.1、页和页表(page & page_table)

1.1.1、页(page)

1.1.2、页表(page_table)

1.1、页和页表(page & page_table)

1.1.1、页(page)

页的结构在/src/storage/page.h中有定义

```
1 class Page {
2     friend class BufferPoolManager;
3
4     public:
5
6     Page() { reset_memory(); }
7
8     ~Page() = default;
9
10    PageId get_page_id() const { return id_; }
11
12    inline char *get_data() { return data_; }
13
14    bool is_dirty() const { return is_dirty_; }
15
16    static constexpr size_t OFFSET_PAGE_START = 0;
17    static constexpr size_t OFFSET_LSN = 0;
18    static constexpr size_t OFFSET_PAGE_HDR = 4;
19
20    inline lsn_t get_page_lsn() { return *reinterpret_cast<lsn_t *>(get_data() + OFFSET_LSN) ; }
21
22    inline void set_page_lsn(lsn_t page_lsn) { memcpy(get_data() + OFFSET_LSN, &page_lsn, sizeof(lsn_t)); }
23
24    private:
25    void reset_memory() { memset(data_, OFFSET_PAGE_START, PAGE_SIZE); }
26    // 将data_的PAGE_SIZE个字节填充为0
27
28    /** page的唯一标识符 */
29    PageId id_;
30
31    /** The actual data that is stored within a page.
32     * 该页面在bufferPool中的偏移地址
33     */
34    char data_[PAGE_SIZE] = {};
35
36    /** 脏页判断 */
37    bool is_dirty_ = false;
38
39    /** The pin count of this page. */
40    int pin_count_ = 0;
41};
```

- 初始化页调用reset_memory()函数。

- 包含的数据

```
1  /** page的唯一标识符 */
2      PageId id_;
3
4  /** The actual data that is stored within a page.
5      * 该页面在bufferPool中的偏移地址
6      */
7      char data_[PAGE_SIZE] = {};
8
9      /** 脏页判断 */
10     bool is_dirty_ = false;
11
12     /** The pin count of this page. */
13     int pin_count_ = 0;
```

包含的数据有1、页面id；2、页面数据；3、脏页标志；4、pin计数。

同时还包含了一个pageId的结构体，这段数据结构起了记录page头信息的作用

```
1  struct PageId {
2      int fd; // Page所在的磁盘文件开启后的文件描述符，来定位打开的文件在内存中的位置
3      page_id_t page_no = INVALID_PAGE_ID;
4
5      friend bool operator==(const PageId &x, const PageId &y) { return x.fd == y.fd && x.page_no == y.page_no; }
6      bool operator<(const PageId& x) const {
7          if(fd < x.fd) return true;
8          return page_no < x.page_no;
9      }
10
11     std::string toString() {
12         return "{fd: " + std::to_string(fd) + " page_no: " + std::to_string(page_no) + "}";
13     }
14
15     inline int64_t Get() const {
16         return (static_cast<int64_t>(fd << 16) | page_no);
17     }
18 };
```

包含的数据有：1、fd文件描述符；2、页面编号(这才是id号)；

1.1.2、页表(page_table)

在rucbase里，在/src/storage/buffer_pool_manager.h文件中。仅用存储帧id、页面id的关联关系。

```
private:
    size_t pool_size_;           // buffer_pool中可容纳页面的个数，即帧的个数
    Page *pages_;               // buffer_pool中的Page对象数组，在构造空间中申请内存空间，在析构函数中释放，大小为BUFFER_POOL_SIZE
    std::unordered_map<PageId, frame_id_t, PageIdHash> page_table_; // 帧号和页面号的映射哈希表，用于根据页面的PageId定位该页面的帧编号
    std::list<frame_id_t> free_list_; // 空闲帧编号的链表
    DiskManager *disk_manager_;
    Replacer *replacer_;        // buffer_pool的置换策略，当前赛题中为LRU置换策略
    std::mutex latch_;          // 用于共享数据结构的并发控制
```

其中PageIdHash在文件/src/storage/page.h中有定义

```
// PageId的自定义哈希算法，用于构建unordered_map<PageId, frame_id_t, PageIdHash>
struct PageIdHash {
    size_t operator()(const PageId &x) const { return (x.fd << 16) | x.page_no; }
};
```

▼ PageIdHash

C++ |

```
1 // PageId的自定义哈希算法，用于构建unordered_map<PageId, frame_id_t, PageIdHash
  >
2 struct PageIdHash {
3     size_t operator()(const PageId &x) const { return (x.fd << 16) | x.page
  _no; }
4 };
```

该哈希函数将 fd 的位向左移动 16 个位置并和page_no进行按位与的操作。